DAY-06

'OPERATORS'

Operators in JAVA.

① Arithmatic operators.

(+, -, *, /, %

+=, -=, *=, /=, %=)

++, --)

post Increment and pre Increment / decrement.

```
int a = 7.
 a++;      // 8   post increment
++a;       // 9   pre increment
   // a = 9
int b = a++;    // b = a ; a = a+1;
System.out.println( a + ", " + b);
            // 10 , 9

int c = ++a ; //c = a+1;
 System.out.println(a+", "+ c);
      // 11, 11
```

② Relational operators ;

(==, !=, <, >, <=, >=) Result Boolean value.

```
int a = 5;
int b = 10;
```

Relational

a == b; is a is equal to b ?

$\boxed{a = b;}$ assign value of b to a.

Assignment

```
boolean c = (a==b);
 System.out.println(c);
```

(Expression)**

$\boxed{a != b;}$ Is a not equal to b ?

```
c = (a != b);
System.out.println(c);
```

boolean c = (a<b) is a<b ? T / F

c = (a>b) T / F

c = (a<=b); T / F.

c = (a>=b) T / F.

③ Bitwise operators

Bit manipulation: unary operator. (need only one operand)

{ &, |, ^, ~, >>, <<, >>>,
&=, |=, ^=, >>=, <<=, >>>= }

```
int a = 2;
int b = 3;

int c = a & b;
```

$(2)_{10} \rightarrow (10)_2$
$(3)_{10} \rightarrow (11)_2$

take byte to understand concept.

```
byte a = 2;
byte b = 3;

byte c = a & b;
```

a = 0 0 0 0 0 0 1 0
256 128 64 32 16 8 2 1

Truth Table

ex.

AND OR

A	B	A & B	A | B	A ^ B	~A	~B
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	0	1
1	1	1	1	0	0	0

{ ^ — odd no. of 1's result → 1
even no. of 1's result → 0 } #

c = c & 2;
c &= 2; (or)

Left shift (<<)

```
byte b = 8;      0 0 0 0 1 0 0 0 = 8.
b = b<<1;
           0  0 0 0 1 0 0 0 0
```

final output
=> 16

b << 1 ≅ b×2
b << 2 ≅ $b \times 2^2$

128 64 32 16 8 4 2 1

b = 8 : 0 0 0 0 1 0 0 0 = 8

b = b<<1 0 0 0 0 1 0 0 0 0 = 16

b = b<<1 0 0 1 0 0 0 0 0 = 32

b = b<<1 0 1 0 0 0 0 0 0 = 64

b = b<<1 1 0 0 0 0 0 0 0 = -128

8 bits → 6^{th} bits (+ve)
↳ 7^{th} bits (−ve)

n bits → (n−2) bits (+ve)
↳ (n−1) bits (−ve)

b = b<<1 1 0 0 0 0 0 0 0 0 = 0

Type promotion : → (>>, <<)
↳ byte, short → int
↳ int long (✓)
↳ byte, short (×)

byte b=1 0 0 0 0 0 0 0 1 = 1

b = b<<7 1 0 0 0 0 0 0 0 = -128

⇓

[b = (byte) b << 7]

+ve MSB

[0] [ ] [ ] [1 0 0 0 0 0 0 0] = +128

8 8 8 8

32 bits

[b = (byte) b<<1] 1 [0 0 0 0 0 0 0 0] = 0

int

byte

int

[(1 1 1 1 ... 1)(1 0 0 0 0 0 0 0)]

24 times 8 bits

[(1 1 1 ... 1) 0 0 0 0 0 0 0]

int i = 1;

i<<1; [ ] [ ] [ ] [0 0 0 0 0 0 0 1]

31bit

0th bit

i<<30; [0100 0000] [ ] [ ] [0]

[i<<31] → [1 0]

-ve

[Integer Min-value]

i<<32 x possible

[int]

byte short

[x % 32]

i<<31 = i<<(31%32);

% 32

(0 — 31)

[i<<32 = i<<0 = i]

[i<<33 = i<<1]

Right shift

byte b = 8 [0] 0 0 0 1 0 0 0

b = b>>1 (1) [0] 0 0 0 0 1 0 0 = 4

(byte)

MSB

Negative No's

1 0 0 0 0 0 0 0

1 0 0 0 0 0 0 0

byte b = -128 1 0 0 0 0 0 0 0 = -128

[1 1 0 0 0 0 0 0] 0 -64

int i = 128;

④ Shift Right with zeroes (>>>)
(Dumb shift)

byte b=-128; $\boxed{1}$ 0 0 0 0 0 0 0 = -128

b>>1; $\boxed{1}$ 1 0 0 0 0 0 0 = -64

b>>>1 0 1 0 0 0 0 0 0 = +64

Logical operators ; (work on expression)

① && (AND)
② || (OR)
③ !

int a = 5;
int b = 10;
int c = 15;

boolean ~~int~~ d = (a<b) && (a<c);

A	B	A && B	A || B
F	F	F	F
F	T	F	T
T	F	F	T
T	T	T	T

Short circuit
↳ &&

$\boxed{A}$ && B
|
F

$\boxed{A}$ || B
|
T

Bitwise operator.
↳ &
↳ |

int a = 5
b = 10
c = 15
d = (a<b) & (a<c);

$\boxed{\text{Does not do short-circuiting.}}$

Assignment operator:

int a = 5; // Assignment.

int a = b = c = 10; (✓)

Ternary operator:
(if else time)

Operator precedence:

(left to right)

- use braces to self. operate.